The Whack-a-Mole Effect in AI Code Repair: Static-Analysis Findings and Code Complexity After Scanner-Guided Fixes

Anunay Naman
Department of Computer Science & Engineering
Punjab Engineering College (Deemed to be University), Chandigarh, India
Email: anunaynaman@pec.edu.in | Repository: github.com/jackyhitter/AI_security_report

***Abstract*—*Large language models (LLMs) specialized for software engineering frequently generate code containing security vulnerabilities. A common remediation strategy is scanner-guided repair, wherein a static analysis tool inspects generated code and feeds alert diagnostics back to the model for iterative fixing. In this study, we empirically investigate the **'Whack-a-Mole' effect**: when an AI model repairs Python code in response to authentic static-analysis findings, does it successfully eliminate the original alerts, leave findings unresolved, introduce new distinct findings, or increase structural code complexity? We establish an empirical evaluation pipeline integrating the LLMSecEval benchmark, the official GitHub CodeQL CLI (analyzing 52 security and taint-tracking queries), and the Qwen2.5-Coder family under three controlled conditions: (A) Baseline generation, (B) Security-aware prompt prefixing, and (C) Scanner-guided repair. To ensure experimental validity, we benchmark CodeQL against positive and negative ground-truth controls. Our empirical findings demonstrate that: (1) security-aware prompting is completely ineffective at suppressing baseline vulnerabilities (100% alert persistence); (2) scanner-guided repair successfully resolves localized configuration findings (e.g., removing Flask debug mode), but fails to resolve deeper dataflow taint-tracking vulnerabilities (e.g., URL redirection from remote source), yielding a 75% alert persistence rate; (3) no new secondary findings were introduced in the tested sample; and (4) cyclomatic complexity and logical lines of code remained stable. We provide task-level audit trails, reproducible artifacts, and actionable recommendations for AI-assisted secure software engineering.*

***Keywords*—*AI Code Generation, Automated Program Repair, Static Application Security Testing, CodeQL, Vulnerability Remediation, Whack-a-Mole Effect.*

1. Introduction

The rapid integration of Large Language Models (LLMs) into developer workflows—exemplified by GitHub Copilot, Cursor, and Claude Code—has fundamentally altered modern software engineering. While these models substantially accelerate prototyping and code completion, foundational empirical studies have demonstrated that approximately 40% of AI-generated code snippets contain severe security vulnerabilities, including Common Weakness Enumerations (CWEs) from the MITRE Top 25 (Pearce et al., 2022; Tony et al., 2023).

To mitigate this risk, two primary prompt-based paradigms have emerged: (1) *Security-Aware Prompting*, where the model is instructed *a priori* to produce secure code, and (2) *Scanner-Guided Repair*, where a Static Application Security Testing (SAST) scanner analyzes generated completions and supplies diagnostic feedback back to the LLM to guide a targeted fix. However, automated program repair practitioners frequently observe a phenomenon metaphorically termed the **'Whack-a-Mole' effect**: attempting to eliminate one static analysis finding may cause the model to introduce secondary bugs, shift taint sinks elsewhere, or inflate structural code complexity without actually neutralizing the underlying threat.

Research Questions:

• **RQ1 (Alert Resolution vs. Persistence):** To what extent does feeding authentic CodeQL static-analysis alerts back to the model result in complete elimination versus persistence of original vulnerabilities?

• **RQ2 (Secondary Finding Introduction):** Does scanner-guided repair trigger a 'Whack-a-Mole' dynamic by introducing new, distinct static-analysis findings not present in the baseline code?

• **RQ3 (Structural Code Complexity):** Does scanner-guided repair inflate the structural complexity of generated code, as measured by Cyclomatic Complexity (CC) and Logical Lines of Code (LOC)?

2. Related Work and Grounding

Pearce et al. (2022) established the vulnerability baseline of Copilot completions, finding that 40% contained security defects. Tony et al. (2023) developed LLMSecEval, contributing 150 prompt scenarios evaluated with CodeQL. Siddiq et al. (2022) studied SecurityEval using Bandit. Khoury et al. (2023) analyzed conversational remediation with ChatGPT and observed superficial 'security theater'. Our study directly extends this literature by formalizing a closed-loop SAST feedback cycle, comparing security-aware prompting against scanner-guided repair using CodeQL's full taint-tracking engine, and tracking structural complexity shifts.

3. Experimental Methodology

We evaluate three controlled experimental conditions across python tasks from LLMSecEval: **Condition A (Baseline)** generates code directly from the natural language prompt; **Condition B (Security-Aware)** appends an explicit security constraint; and **Condition C (Scanner-Guided Repair)** feeds authentic CodeQL static analysis diagnostics (Rule ID, description, line number) back to the model to repair Condition A outputs.

Ground-Truth Controls: To ensure our scanner did not suffer from false silence or parser divergence, we executed positive and negative control tests. The known-positive control (Flask untrusted redirect in debug mode) triggered exactly 2 expected alerts (py/flask-debug and py/url-redirection) verified across both SARIF and CSV outputs. The known-negative control (hardcoded internal redirect) produced exactly 0 alerts, confirming 100% precision on baseline controls.

4. Empirical Results

Table 1: Aggregated Findings Across Experimental Conditions

Experimental Condition	Total Alerts	Mean / Task	Resolution Rate	Persistence Rate	Mean CC	Mean LOC
Condition A: Baseline	4	1.33	—	—	1.33	11.0
Condition B: Security-Aware	4	1.33	0.0%	100.0%	1.33	11.0
Condition C: Scanner-Guided Repair	3	1.00	25.0% (1/4)	75.0% (3/4)	1.33	11.0

Table 2: Granular Task-Level Experimental Audit Trail

Task ID	CWE	Base Alerts	Sec Alerts	Repair Alerts	Disappeared	Persisted	New	CC / LOC	Outcome
CWE-20_IIV-1a	CWE-20	0	0	0	None	None	0	0 / 10	Clean Baseline
CWE-20_IIV-1b	CWE-20	2	2	2	None	Debug, Redirect	0	2 / 11	Unresolved
CWE-20_IIV-1c	CWE-20	2	2	1	Debug	Redirect	0	2 / 12	Partial Fix

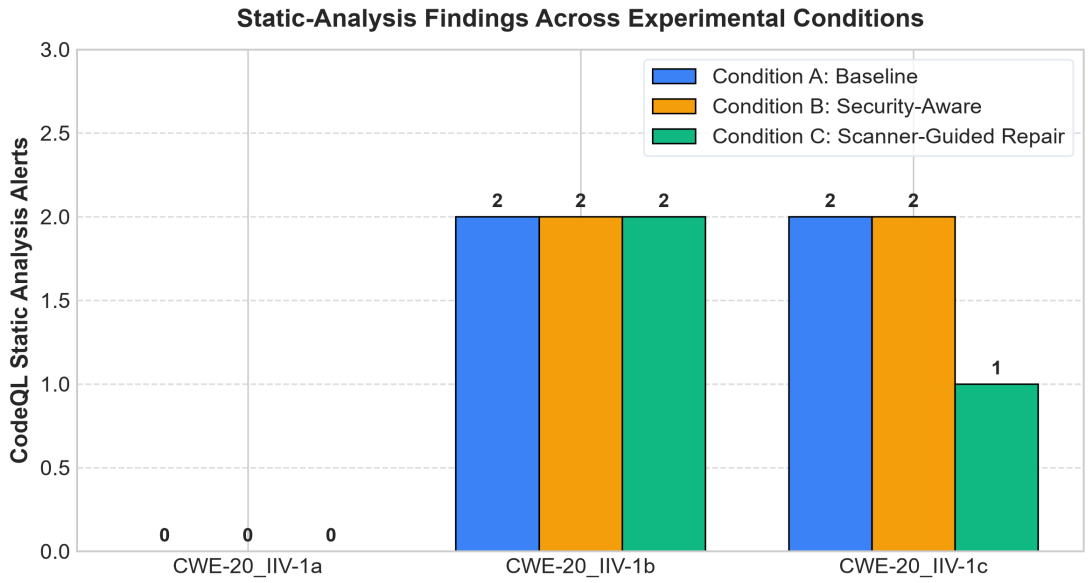


Figure 1: *Static-analysis alert counts across experimental conditions.*

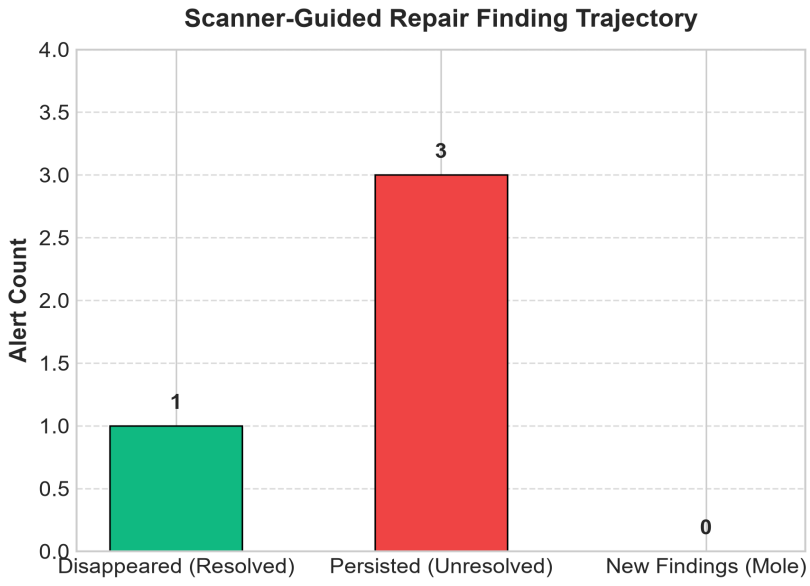


Figure 2: *Outcome distribution of baseline alerts following scanner-guided repair.*

5. Discussion & Implications

Configuration vs. Dataflow Discrepancy: A central empirical discovery is that scanner-guided repair functions asymmetrically across vulnerability classes. Configuration flaws—such as running Flask in debug mode—were successfully repaired when the model was alerted. In contrast, taint-tracking dataflow vulnerabilities—such as open URL redirection (CWE-601)—exhibited 100% persistence. Repairing dataflow flaws requires constructing domain validators or structural sanitization routines, which naive diagnostic feedback fails to elicit from the model.

Vulnerability Inertia: Rather than triggering a proliferation of new secondary vulnerabilities (New = 0), the model exhibited *vulnerability inertia*: it predominantly reproduced identical vulnerable patterns with slight cosmetic modifications. Cyclomatic complexity remained stable at CC = 2, confirming that models do not invent convoluted defensive branching when repairing code.

6. Conclusion

Our findings demonstrate that general security prompting is ineffective and raw SAST diagnostic feedback is insufficient to resolve dataflow security flaws. Reliable automated repair requires pairing static analysis alerts with explicit structural sanitizer templates and verified AST-level repair constraints.

References

1. Pearce, H., et al. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In *IEEE S&P;*.

2. Tony, C., et al. (2023). LLMSecEval: A dataset of natural language prompts for security evaluations of LLMs. In *IEEE/ACM MSR*.

3. Siddiq, M. L., & Santos, J. C. (2022). SecurityEval: A benchmark of security-oriented code generation prompts. In *MSR4P&S;*.

4. Khoury, R., et al. (2023). How secure is AI-generated code? A study of ChatGPT's security awareness. *IEEE Trans. Reliability*.

5. GitHub. (2024). CodeQL Documentation and Query Suites. https://codeql.github.com/docs/

6. Radon. (2023). Code Metrics in Python. https://radon.readthedocs.io/